



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 2. Математические операции

Содержание лекции

В данной главе будут рассмотрены следующие темы:

-  Целые числа
-  Вещественные числа
-  Арифметические операции
-  Приоритет арифметических операций
-  Операции сравнения
-  Числа в различных системах счисления
-  Встроенные математические функции
-  Модуль math
-  Модуль random

Числа в математике



Числа в Python

3 типа чисел в Python (встроенные типы):

- ⦿ Целые
- ⦿ Вещественные
- ⦿ Комплексные

Системы счисления:

- ⦿ Любое основание от 2 до 36
- ⦿ Имеются функции преобразований

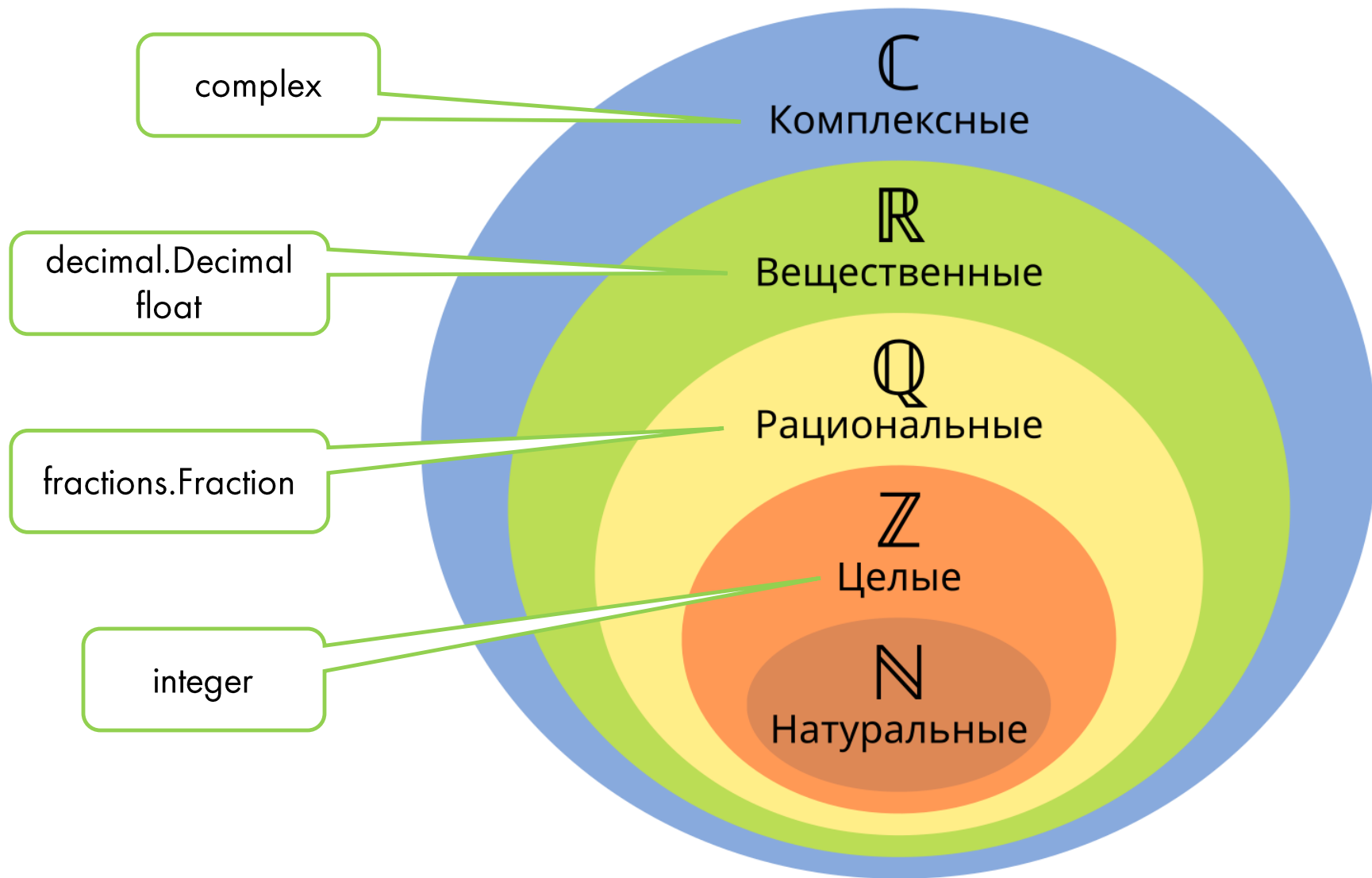
Автоматическое распознавание типа числа

- ⦿ Нет необходимости явно указывать тип числа

Специальные типы (стандартные библиотеки)

- ⦿ Рациональные числа
- ⦿ Десятичные дроби
- ⦿ Случайные величины

Поддержка чисел различных типов



Целые числа

📡 Целые числа – обычные целые числа со знаком или без

📡 Запись

- ⦿ Используются десятичные цифры
- ⦿ Не могут начинаться с нуля
- ⦿ Знаки + и – являются унарными операторами

📡 Длинная арифметика

- ⦿ Доступна по умолчанию
- ⦿ Необходима в некоторых разделах математики

```
0
15
75384748327837257111764874480574356743982183458735233
```

015 – Ошибка!

Вещественные числа

- Вещественные числа – числа со знаком «.»
 - «.» отделяет целую часть от дробной
- Запись
 - Можно не указывать целую или дробную часть
 - Существует экспоненциальная форма записи
- Длинная арифметика не поддерживается
- Погрешность и ограниченная точность!

2.0

2. = 2.0

.5 = 0.5

2e3 = 2.0 * 10 ** 3 = 2000.0

Особенности вещественных чисел

Погрешность и ограниченная точность

```
2 ** (1/2) = 1.4142135623730951
```

Неточности вычислений

```
(2 ** (1/2)) ** 2 = 2.0000000000000004
```

inf, -inf и OverflowError

```
10e1000 = inf
```

```
-10e1000 = -inf
```

```
10.0 ** 1000 = OverflowError
```


Арифметические операции (1 из 3)

 Простые операции (сложение, вычитание, умножение, деление)

$x + y$

$x - y$

$x * y$

x / y

 Унарные операторы (+ и -)

$+a$

$-a$

$++a$ – Ошибка! Для инкремента используется оператор $+=$

$--a$ – Ошибка! Для декремента используется оператор $-=$

Арифметические операции (2 из 3)

Деление нацело и остаток от деления

- ⦿ `//` - деление нацело
- ⦿ `%` - нахождение остатка

`x // y`

`12 // 5 = 2`

`(-1) // 2 = -1`

`x % y`

`3 % 2 = 1`

`4 % 2 = 0`

`153 % 10 = 3`

`divmod(x, y) = (x // y, x % y)`

Арифметические операции (3 из 3)

Возведение в степень

`x ** y`

`2 ** 4 = 16`

`pow(x, y[, z])`

`pow(2, 4) = 2 ** 4 = 16`

`pow(2, 4, 3) = (2 ** 4) % 3 = 1`

Получение модуля числа

`abs(x)`

`abs(-10) = 10`

Преобразование типов

Операции преобразования

- ◉ `int(x)` – преобразование к целочисленному типу
- ◉ `float(x)` – преобразование к вещественному типу числа

Автоматическое преобразование к более общему типу

- ◉ Схема преобразований: `int -> float -> complex`

Если один из операндов является вещественным числом, то результат операции преобразуется к вещественному типу

```
int(10) = 10
```

```
int(12.8) = 12
```

```
float(12.8) = 12.8
```

```
float(10) = 10.0
```

```
10 / 5 = 2.0
```

```
10 + 2 = 12
```

```
10 + 5.0 = 15.0
```

Встроенные математические функции

- 📶 `abs(x)` – получение модуля числа
- 📶 `pow(x, y[, z])` – возведение в степень
- 📶 `divmod(x, y)` – деление нацело и остаток от деления
- 📶 `round(x[, n])` – округление числа
- 📶 `min()`, `max()`, `sum()` – мин., макс. элемент и сумма элементов
- 📶 `oct(x)`, `bin(x)`, `hex(x)` – перевод в различные системы счисления

```
round(2.5) == 2
round(2.123, 2) == 2.12
round(2.8, 3) == 2.8
```

```
a = [1, 2, 5]
min(a) == 1
max(1, 2, 5) == 5
sum(a) == 8
```

Обратите внимание на
количество знаков в
дробной части

Приоритет арифметических операций

 Приоритеты соответствуют приоритетам операций в математике

⦿ Расположены в порядке убывания

$x^{**}y$	Возведение в степень
$+x, -x$	Изменение знака
$x \% y$ $x // y$ x / y $x * y$	Взятие остатка Целочисленное деление «Обычное» деление Умножение
$x - y$ $x + y$	Вычитание Сложение

Операции сравнения

- 🔊 $x < y$ – True, если x меньше y
- 🔊 $x \leq y$ – True, если x меньше или равно y
- 🔊 $x > y$ – True, если x больше y
- 🔊 $x \geq y$ – True, если x больше или равно y
- 🔊 $x == y$ – True, если x равно y
- 🔊 $x != y$ – True, если x не равно y
- 🔊 $x \text{ is } y$ – True, если x и y – это один объект
- 🔊 $x \text{ is not } y$ – True, если x и y – это разные объекты

Приоритет операций сравнения ниже, чем у арифметических операций

```
x = 5
y = 10
```

```
x < y - True
x <= y - True
x > y - False
x >= y - False
x == y - False
x != y - True
```

```
x_2 = x
x is x_2 - True
y = y - 5
x is not y - True
```

Системы счисления (1 из 3)

📡 Система счисления – способ записи числа с помощью определенного набора символов

📡 Двоичная

- ⦿ Префикс 0b или 0B
- ⦿ Цифры 0 и 1

0b0, 0b1, 0B01110
(0, 1, 14)

📡 Восьмеричная

- ⦿ Префикс 0o или 0O
- ⦿ Цифры от 0 до 7 (включая)

0o1234567, 0o442, 0O43
(342391, 290, 35)

📡 Шестнадцатеричная

- ⦿ Префикс 0x или 0X
- ⦿ Цифры от 0 до 9 (включая)
- ⦿ Буквы A, B, C, D, E, F

0xA2, 0x12, 0XFA
(162, 18, 250)

Системы счисления (2 из 3)

Функции преобразований:

- `bin(x)` – целое число в двоичную
- `oct(x)` – целое число в восьмеричную
- `hex(x)` – целое число в шестнадцатеричную
- `int(x[, s])` – перевод целого числа, записанного в системе счисления с основанием `s`, в десятичную систему счисления

```
x = 22
```

```
bin(x) = 0b10110
```

```
oct(x) = 0o26
```

```
hex(x) = 0x16
```

```
y = '01234'
```

```
int(y, 5) = 194
```

Системы счисления (3 из 3)

- 📶 Для представления вещественного числа в системе счисления, отличной от десятичной, необходимо отдельно представить в данной системе счисления целую и дробную части
- 📶 Встроенных методов не существует
- 📶 Возможный алгоритм:
 1. Разделить десятичное число на целую и дробную части
 2. Перевести каждую часть в нужную систему счисления
 3. У дробной части убрать префикс
 4. Соединить полученные части

1. 123.456
2. 123 = **0b**1111011
3. 456 = **0b**111001000
4. 123.456 = **0b**1111011.111001000

Модуль math (1 из 6)

- Модуль math – встроенный модуль Python, предоставляющий популярные математические функции
- Модуль содержит:
 - Константы
 - Логарифмические функции
 - Тригонометрические функции
 - Прочие арифметические функции
- Импорт модуля
 - Импорт модуля целиком
 - Импорт отдельных атрибутов модуля

```
import math  
print(math.e)
```

Вывод: 2.718281828459045

Модуль math (2 из 6)

 Модуль math содержит несколько констант:

⦿ `math.pi` – число π (Пи)

3.141592653589793

⦿ `math.e` – экспонента (e)

2.718281828459045

⦿ `math.inf` – положительная бесконечность

⦿ `math.nan` – не число (not a number)

```
import math
from math import e, pi

circle_area = 2 * pi * 5
variable_1 = e * 2
infinity = -math.inf
not_a_number = math.nan
```

Модуль math (3 из 6)

📡 Для проверки чисел используются следующие функции:

- ⦿ `isinf(x)` – возвращает `True`, если `x` – бесконечность
- ⦿ `isfinite(x)` – возвращает `True`, если `x` – конечное число
- ⦿ `isnan(x)` – возвращает `True`, если `x` – не число

```
import math
```

```
var_1 = math.inf
```

```
var_2 = math.nan
```

```
print(math.isinf(var_1)) - Вывод: True
```

```
print(math.isfinite(var_1)) - Вывод: False
```

```
print(math.isnan(var_2)) - Вывод: True
```

Модуль math (4 из 6)

Логарифмические функции модуля math:

- ⦿ `math.log(x[, n])` – вычисление логарифма числа x по основанию n . Если n не указано, то вычисляется натуральный логарифм
- ⦿ `math.log10(x)` – вычисление десятичного логарифма числа x
- ⦿ `math.log2(x)` – вычисление двоичного логарифма числа x

Функции `math.log10(x)` и `math.log2(x)` точнее, чем функции `math.log(x, 10)` и `math.log(x, 2)`

```
import math
```

```
print(math.log(16, 4)) - Вывод: 2.0
```

```
print(math.log(math.e ** 2)) - Вывод: 2.0
```

```
print(math.log10(100)) - Вывод: 2.0
```

```
print(math.log2(8)) - Вывод: 3.0
```

Модуль math (5 из 6)

Простые тригонометрические функции:

- ⦿ `math.sin(x)` – возвращает синус угла x , заданного в радианах
- ⦿ `math.cos(x)` – возвращает косинус угла x , заданного в радианах
- ⦿ `math.tan(x)` – возвращает тангенс угла x , заданного в радианах

Обратные тригонометрические функции:

- ⦿ `math.asin(x)` – возвращает арксинус значения x
- ⦿ `math.acos(x)` – возвращает арккосинус значения x
- ⦿ `math.atan(x)` – возвращает арктангенс значения x

```
from math import sin, asin, pi
print(sin(pi/2)) – Вывод: 1.0
print(asin(1)) – Вывод: 1.5707963267948966
```

Модуль math (6 из 6)

Прочие функции:

- ⦿ `math.ceil(x)` – округление числа `x` «вверх»
- ⦿ `math.floor(x)` – округление числа `x` «вниз»
- ⦿ `math.trunc(x)` – взятие только целой части числа
- ⦿ `math.sqrt(x)` – вычисление квадратного корня из `x`
- ⦿ `math.pow(x, y)` – возведение числа `x` в степень `y`
- ⦿ `math.fabs(x)` – получение абсолютного значения числа (модуля)
- ⦿ `math.factorial(x)` – получение факториала числа

```
import math
print(math.ceil(2.3)) - Вывод: 3
print(math.sqrt(4)) - Вывод: 2.0
print(math.factorial(5)) - Вывод: 120
```


Модуль decimal (1 из 2)

- 🌀 Модуль `decimal` – встроенный модуль Python, реализующий точную десятичную арифметику с плавающей точкой
 - 🕒 `Context` определяет настройки операций, такие как точность, правила округления, допустимый диапазон показателей степени

```
from decimal import *
getcontext()
# Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999,
# Emax=999999, capitals=1, clamp=0, flags=[], traps=[Overflow,
# DivisionByZero, InvalidOperation])

getcontext().prec = 6
Decimal(1) / Decimal(7) # Decimal('0.142857')
getcontext().prec = 28
Decimal(1) / Decimal(7)
# Decimal('0.1428571428571428571428571428571429')
```

Модуль decimal (2 из 2)

```
getcontext().prec = 28
Decimal(10) # Decimal('10')
Decimal('3.14') # Decimal('3.14')
Decimal(3.14)
# Decimal('3.1400000000000000124344978758017532527446746826171875')
Decimal((0, (3, 1, 4), -2)) # Decimal('3.14')
Decimal(str(2.0 ** 0.5)) # Decimal('1.4142135623730951')
Decimal(2) ** Decimal('0.5')
# Decimal('1.414213562373095048801688724')
Decimal('NaN') # Decimal('NaN')
Decimal('-Infinity') # Decimal('-Infinity')
getcontext().prec = 6
Decimal('3.0') # Decimal('3.0')
Decimal('3.1415926535') # Decimal('3.1415926535')
Decimal('3.1415926535') + Decimal('2.7182818285')
# Decimal('5.85987')
getcontext().rounding = ROUND_UP
Decimal('3.1415926535') + Decimal('2.7182818285')
# Decimal('5.85988')
```

Модуль fractions (1 из 2)

🌀 Модуль `fraction` – встроенный модуль Python, реализующий рациональные числа (простые дроби)

```
from fractions import Fraction
Fraction(16, -10) # Fraction(-8, 5)
Fraction(123) # Fraction(123, 1)
Fraction() # Fraction(0, 1)
Fraction('3/7') # Fraction(3, 7)
Fraction(' -3/7 ') # Fraction(-3, 7)
Fraction('1.414213 \t\n') # Fraction(1414213, 1000000)
Fraction('-.125') # Fraction(-1, 8)
Fraction('7e-6') # Fraction(7, 1000000)
Fraction(2.25) # Fraction(9, 4)
Fraction(1.1) # Fraction(2476979795053773, 2251799813685248)
from decimal import Decimal
Fraction(Decimal('1.1')) # Fraction(11, 10)
```

Модуль fractions (2 из 2)

- Значения числителя и знаменателя доступны через соответствующие свойства объекта

```
r = Fraction(1, 3)
r.numerator # 1
r.denominator # 3
```

- Максимальное значение знаменателя можно ограничить
 - Аппроксимация float значений рациональными числами
 - Восстановление рациональных значений после float вычислений

```
Fraction('3.1415926535897932').limit_denominator(1000)
# Fraction(355, 113)
from math import pi, cos
Fraction(cos(pi/3)) # Fraction(4503599627370497, 9007199254740992)
Fraction(cos(pi/3)).limit_denominator() # Fraction(1, 2)
Fraction(1.1).limit_denominator() # Fraction(11, 10)
```

Модуль random (1 из 4)

- 🔊 Модуль random – встроенный модуль Python, предоставляющий функции для работы со случайными объектами
- 🔊 Модуль содержит:
 - ⦿ Генераторы чисел
 - ⦿ Функции для работы с последовательностями
 - ⦿ Генераторы распределений
- 🔊 Импорт модуля
 - ⦿ Импорт модуля целиком
 - ⦿ Импорт отдельных атрибутов модуля

```
import random  
print(random.random())  
Вывод: 0.5485684602706552
```

Модуль random (2 из 4)

- 📡 Для генерации случайных чисел используются функции:
- ⦿ `random.randint(a, b)` – случайное число x , $a \leq x \leq b$
 - ⦿ `random.uniform(a, b)` – случайное вещественное число x , $a \leq x \leq b$
 - ⦿ `random.random()` – случайное число от 0 до 1

```
import random
```

```
rand_1 = random.randint(1, 10)
```

```
rand_2 = random.uniform(1, 10)
```

```
rand_3 = random.random()
```

```
print(rand_1) – Вывод: 4
```

```
print(rand_2) – Вывод: 8.545672691021313
```

```
print(rand_3) – Вывод: 0.10920281392310593
```

Модуль random (3 из 4)

 Функции для работы с последовательностями:

- ⦿ `random.choice(x)` – случайный элемент последовательности `x`
- ⦿ `random.shuffle(x)` – случайное перемешивание последовательности `x`
- ⦿ `random.sample(x, n)` – возвращает список из `n` элементов, составленный из элементов последовательности `x`

```
import random
```

```
arr = [1, 2, 5, 10]  
print(random.choice(arr)) - Вывод: 2
```

```
random.shuffle(arr)  
print(arr) - Вывод: [2, 10, 5, 1]
```

```
sample_arr = random.sample(arr, 2)  
print(sample_arr) - Вывод: [1, 5]
```

Модуль random (4 из 4)

- 📡 Функция `random.seed(param)` позволяет влиять на инициализацию генератора случайных чисел
- 📡 Если передавать один и тот же параметр, то будут генерироваться одни и те же случайные числа
- 📡 По умолчанию в качестве параметра используется системное время

```
from random import seed, random

seed(20)
print(random()) - Вывод: 0.9056396761745207
print(random()) - Вывод: 0.6862541570267026

seed(20)
print(random()) - Вывод: 0.9056396761745207
print(random()) - Вывод: 0.6862541570267026
```

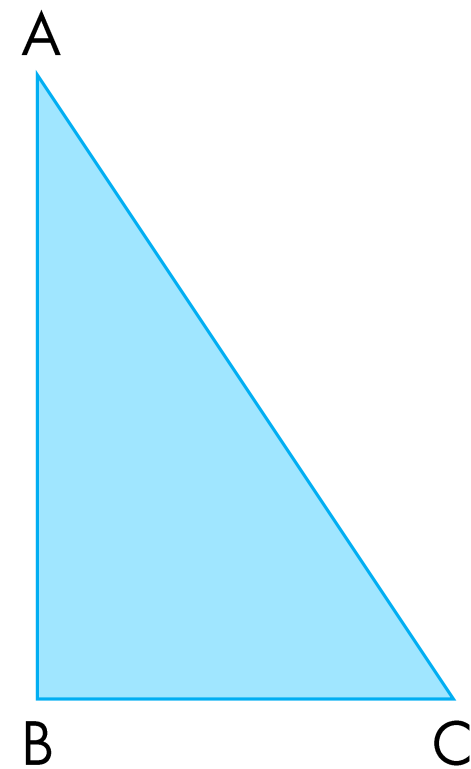
Генерируются
одинаковые
последовательности
случайных чисел

Примеры (1 из 2)

📶 Школьная геометрическая задача:

```
from math import sqrt, pow, sin, radians
cath_ba = 3
cath_bc = 4
hyp_ac = sqrt(cath_ba ** 2 + pow(cath_bc, 2))
print(hyp_ac)                                Вывод: 5.0
```

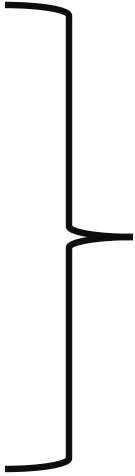
```
cath_ba = 3
hyp_ac = 6
angle_acb = 30
sin_angle_acb = sin(radians(angle_acb))
print(sin_angle_acb)                        Вывод: 4.99..4
sin_angle_acb = round(sin_angle_acb, 2)
print(sin_angle_acb)                        Вывод: 0.5
```



Примеры (2 из 2)

Случайные числа:

```
from random import randint
age = [
    randint(60, 90),
    randint(60, 90),
    ...
    randint(60, 90),
    randint(60, 90)
]
```



Список из случайных
продолжительностей
жизни



```
print(age)
print("Максимальный возраст:", max(age))
print("Минимальный возраст:", min(age))
print("Средний возраст:", round(sum(age)/len(age)))
```

Вывод: [68, 63, ..., 87, 60]
Вывод: "... 87"
Вывод: "... 62"
Вывод: "... 74"